DAA UNIT 1

Q1(a) Why correctness of the algorithm is important? Define loop invariant property and prove the correctness of finding summation of n numbers using loop invariant property. (7m)

Algorithm correctness means that an algorithm produces the **correct output for every valid input**.

It is important because:

1. It ensures accurate results.
2. It makes the algorithm reliable.
3. It handles all valid inputs correctly.
4. It avoids incorrect or unexpected results.

Loop Invariant Property - A **loop invariant** is a condition that remains **true before and after every iteration** of a loop.

It is proved using three steps:

1. **Initialization** – Prove that the invariant is true before the first iteration.
2. **Maintenance** – Prove that if it is true before an iteration, it remains true after that iteration.
3. **Termination** – When the loop terminates, use the invariant to prove that the algorithm gives the correct result.

Algorithm to find sum of n numbers

SUM(A, n)

1. sum ← 0

2. i ← 1

3. while i ≤ n do

4. sum ← sum + A[i]

5. i ← i + 1

6. return sum

Loop Invariant - At the beginning of every iteration, sum=A[1]+A[2]+⋯+A[i−1]

1. Initialization: Initially, i = 1 and sum = 0. There are no elements before A[1], so: sum=0 Thus, the loop invariant is true.

2. Maintenance: Assume: sum=A[1]+A[2]+⋯+A[i−1] After executing: sum=sum+A[i] we get: sum=A[1]+A[2]+⋯+A[i] Then i is increased by 1. Therefore, the invariant remains true for the next iteration.

3. Termination: The loop terminates when: i=n+1, Therefore, according to the invariant: sum=A[1]+A[2]+⋯+A[n], Hence, the algorithm correctly calculates the **sum of n numbers**.

Q1(b) Any 4 Problem-Solving Strategies — 8 Marks

1. Brute Force - Brute Force solves a problem by **trying all possible solutions** and selecting the required one. This is straightforward technique with naïve approach.
Example: Linear Search checks each element one by one.

Advantages: Simple and easy to implement.
Disadvantage: Can be inefficient for large inputs.

2. Divide and Conquer - It divides a problem into **smaller subproblems**, solves them and combines their solutions.

It has three steps:

- **Divide**
- **Conquer**
- **Combine**

Example: Merge Sort, Binary Search.

3. Greedy Method - Greedy strategy selects the **best possible choice at each step** with the hope of obtaining the overall optimal solution.

Example: Fractional Knapsack, Kruskal's Algorithm, Prim's Algorithm.

Advantage: Usually simple and efficient.

Disadvantage: A locally best choice may not always give the globally best solution.

4. Dynamic Programming - Dynamic Programming solves **overlapping subproblems** and stores their results so that they can be reused.

It is based on:

- **Overlapping subproblems**
- **Optimal substructure**

Example: 0/1 Knapsack, Fibonacci, Matrix Chain Multiplication.

Advantage: Avoids repeated calculations and improves efficiency.

5. **Backtracking** - This method is based on trial and error. Each time the solution from the set of solutions is picked and checked if it is the accurate solution or not.

Example: N-Queens Problem, Sudoku Solver.

Q2(a) What is an iterative algorithm? Explain iterative algorithm design issues with examples.

An **iterative algorithm** solves a problem by repeatedly executing a set of instructions using **loops** such as for, while, or do-while.

It does not use recursion.

Example: Factorial of n

factorial ← 1

for i ← 1 to n do

  factorial ← factorial × i

return factorial

For n = 5: 1×2×3×4×5=120

Iterative Algorithm Design Issues -

While designing an iterative algorithm, the following issues should be considered:

1. Initialization: Variables must be initialized correctly before the loop starts.

Example: sum ← 0

2. Loop condition: The condition should correctly determine when the loop should stop.

Example: while i ≤ n

3. Loop progress/update: The loop variable must be updated to avoid an infinite loop.

Example: i ← i + 1

4. Termination: The algorithm must eventually terminate after a finite number of iterations.

5. Correctness / Loop invariant: A condition that remains true during every iteration should be identified to ensure correctness.

Example: Sum of n numbers

sum ← 0

```
i ← 1
while i ≤ n do
    sum ← sum + A[i]
    i ← i + 1
```

Here:

- sum ← 0 → initialization
- i ≤ n → loop condition
- i ← i + 1 → progress
- Loop stops when i > n → termination

Q2(b) How to prove that an algorithm is correct? How to prove correctness using counter example?

Proving Algorithm Correctness

An algorithm is **correct** if it produces the expected output for **every valid input**.

Correctness can generally be proved using:

1. **Pre-condition** – Condition that is true before execution.
2. **Post-condition** – Condition that should be true after execution.
3. **Loop Invariant** – Condition that remains true during loop execution.
4. **Initialization, Maintenance and Termination** are used for iterative algorithms.

Proving correctness using Counter Example

A **counter example** is an input for which the algorithm produces an incorrect result.

To disprove an algorithm:

1. Assume the algorithm is correct.
2. Find a valid input.
3. Execute the algorithm for that input.
4. Compare its output with the expected output.
5. If they differ, the algorithm is **not correct**.

Example - Suppose an algorithm claims:

"The maximum element of an array is always the first element."

Consider: A = [5, 8, 3, 2]

The algorithm gives: Maximum=5

But the actual maximum is: Maximum=8

Therefore, [5, 8, 3, 2] is a **counter example**.

Hence, the given algorithm is **incorrect**.

Q1(a) Prove correctness of the algorithm finding minimum of an array.

Given Algorithm:

```
min = A[0]
for i = 1 to n-1
    if A[i] < min
        min = A[i]
return min
```

Loop Invariant - At the beginning of every iteration, min contains the minimum value among the elements A[0] to A[i-1].

1. Initialization: Initially, i = 1 and: min=A[0]

So, among the first element A[0], min is obviously the minimum.

Hence, the loop invariant is **true before the first iteration**.

2. Maintenance

Assume that before an iteration: min=min(A[0],A[1],...,A[i−1])

Now compare A[i] with min.

- If A[i] < min, then min = A[i].
- Otherwise, min remains unchanged.

Therefore, after the iteration: min=min(A[0],A[1],...,A[i])

Thus, the loop invariant remains **true**.

3. Termination

The loop terminates when: i=n

Therefore, according to the loop invariant: min=min(A[0],A[1],...,A[n−1])

Hence, min contains the **minimum element of the entire array**.

Therefore, the given algorithm is **correct**.

Remember: Initialization → Maintenance → Termination ⭐

Q2(a) Prove correctness of the factorial algorithm

Given Algorithm

```
fact(n) {
   if (n == 0)
      return 1;
   else
      return n * fact(n-1);
}
```

We prove correctness using **mathematical induction**.

Statement - We need to prove that: fact(n)=n!

for every n >= 0.

1. Base Case

For: n = 0

The algorithm returns: fact(0) = 1

And by definition: 0! = 1

Therefore: fact(0) = 0!

So, the base case is **true**.

2. Induction Hypothesis

Assume the algorithm correctly calculates the factorial for n-1.

That is: fact(n – 1) = (n – 1) !

3. Induction Step

For n > 0, the algorithm returns: fact(n) = n x fact(n - 1)

Using the induction hypothesis: fact(n) = n x (n – 1) !

We know: n x (n – 1) ! = n!

Therefore: fact(n) = n!

Hence, the algorithm correctly calculates the factorial for n.

Conclusion - Since the **base case is true** and the **induction step is true**, by mathematical induction, the given algorithm correctly computes: n! for every n >= 0.

⭐ **Very important -**

If they give you an **iterative algorithm**, usually think:

Loop Invariant → Initialization → Maintenance → Termination

If they give you a **recursive algorithm**, think:

Base Case → Induction Hypothesis → Induction Step → Conclusion

Q1(a) Given the fastest computer and hypothetically infinite memory, do we still need to study algorithms? Justify. [2 Marks]

Yes, we still need to study algorithms.

Even with the fastest computer and unlimited memory, a **poorly designed algorithm can take much more time and resources** than an efficient one.

For example, an algorithm is much slower than an algorithm for large inputs.

Therefore, algorithms are necessary to achieve **efficiency, correctness, and scalability**.

Q1(b) How can we relate algorithms to technology? Briefly explain. [6 Marks]

Algorithms and technology are closely related because algorithms provide the **steps or logic**, while technology provides the **hardware and software** to execute those steps.

Relationship between Algorithms and Technology:

1. **Software Development:** Algorithms are used to develop applications, operating systems, websites and software.
2. **Artificial Intelligence:** Machine learning and AI use algorithms to learn from data and make decisions.
3. **Database Systems:** Searching, sorting and indexing algorithms help databases retrieve information efficiently.
4. **Computer Networks:** Routing algorithms determine the best path for transferring data between computers.

5. **Security:** Encryption and hashing algorithms are used to protect data and provide secure communication.
6. **Performance Improvement:** Efficient algorithms allow technology to process large amounts of data faster while using fewer resources.

Q1(c) Consider an array A of n integers which are already in sorted order. Let x be the number being searched in the array A in a liner fashion. The code fragment performing this task is given below:

```
int lin _ search (int A []) {
        i=0; flag=0;
        do {
                if (x = = A [i]) then return (1); // Number found
                else i++;
        } while (i < n); .
        return (0); // Number not found
}
```

i) Is this code fragment efficient? Justify.

The given array is **already sorted** and we are using **Linear Search**.

The code checks elements one by one: A[0], A[1], A[2], ..., A[n-1]

Since the array is sorted, we can **stop searching as soon as A[i] > X**.

For example: A = [2, 5, 8, 12, 15, 20] X = 10

When we reach 12, we know that 10 cannot occur after it because the array is sorted.

So the given code is **not fully efficient** even though we are restricted to using linear search.

Improved condition:

```
while (i < n && A[i] <= X) {
   if (A[i] == X)        return 1;
   i++;
}
```

return 0;

Complexity

- **Best case:** ($O(1)$)
- **Worst case:** ($O(n)$)
- **Space:** ($O(1)$)

Thus, the improved linear search is **more efficient** because it uses the sorted property of the array.

ii) Design issue with respect to iterative algorithm

Yes, there is a **design issue related to termination condition**.

In the given algorithm:

```
do {
   ...
   i++;
} while (i < n);
```

The loop does not check whether: $A[i] > X$

Since the array is sorted, the search can be terminated early when: $A[i] > X$

Otherwise, the algorithm unnecessarily continues checking the remaining elements.

Therefore: The iterative algorithm should use an appropriate **loop termination condition** to avoid unnecessary iterations.

⭐ **Final points to remember**

Given code: Linear search → ($O(n)$)

Better linear search for sorted array: Stop when $A[i] > X$

Design issue: Proper **loop termination condition** is important for efficiency.

Q2(b) Consider the following algorithm to find the square of a number:

Given algorithm

```
sqr(n) {
   if (n == 0)
      return 0;
```

```
    else
        return (2n + sqr(n-1) - 1);
}
```

We need to prove: $sqr(n)=n^2$

Proof using Mathematical Induction

1. Base Case

For n = 0 sqr(0) = 0

And: $0^2 = 0$

Therefore, $sqr(0) = 0^2$

So, the base case is **true**. ✓

2. Induction Hypothesis

Assume the algorithm is correct for n - 1

Therefore, $sqr(n-1)=(n-1)^2$

3. Induction Step

For n > 0 the algorithm gives: $sqr(n)=2n+sqr(n-1)-1$

Using the induction hypothesis: $sqr(n)=2n+(n-1)^2-1 =2n+n^2-2n+1-1 = n^2$

Therefore, $sqr(n)=n^2$

Hence, the algorithm correctly calculates the **square of n**. ✓

Since the **base case is true** and the **induction step is true**, by the **principle of mathematical induction**, the given algorithm is correct for all n >= 0.